



PDF Download
3580305.3599530.pdf
03 April 2026
Total Citations: 16
Total Downloads: 1924

Latest updates: <https://dl.acm.org/doi/10.1145/3580305.3599530>

RESEARCH-ARTICLE

TransformerLight: A Novel Sequence Modeling Based Traffic Signaling Mechanism via Gated Transformer

QIANG WU, University of Electronic Science and Technology of China, Chengdu, Sichuan, China

MINGYUAN LI, Beijing University of Posts and Telecommunications, Beijing, Beijing, China

JUN SHEN, University of Wollongong, Wollongong, NSW, Australia

LINYUAN LÜ, University of Science and Technology of China, Hefei, Anhui, China

BO DU, University of Wollongong, Wollongong, NSW, Australia

KE ZHANG, Beijing University of Posts and Telecommunications, Beijing, Beijing, China

Open Access Support provided by:

University of Wollongong

Beijing University of Posts and Telecommunications

University of Electronic Science and Technology of China

University of Science and Technology of China

Published: 04 August 2023

Citation in BibTeX format

KDD '23: The 29th ACM SIGKDD
Conference on Knowledge Discovery and
Data Mining
August 6 - 10, 2023
CA, Long Beach, USA

Conference Sponsors:
SIGMOD
SIGKDD

TransformerLight: A Novel Sequence Modeling Based Traffic Signaling Mechanism via Gated Transformer

Qiang Wu*

Institute of Fundamental and
Frontier Sciences,
University of Electronic Science and
Technology of China,
Chengdu, China
qiang.wu@uestc.edu.cn

Mingyuan Li*

College of Cyberspace Security,
Beijing University of Posts and
Telecommunications,
Beijing, China
henryli_i@bupt.edu.cn

Jun Shen

School of Computing and
Information Technology,
University of Wollongong,
Wollongong, Australia
jshen@uow.edu.au

Linyuan Lü†

School of Cyberspace Security,
University of Science and
Technology of China,
Hefei, China
linyuan.lv@ustc.edu.cn

Bo Du

SMART Infrastructure Facility,
University of Wollongong,
Wollongong, Australia
bdu@uow.edu.au

Ke Zhang

School of Digital Media and Design
Arts,
Beijing University of Posts and
Telecommunications,
Beijing, China
kezhang@bupt.edu.cn

ABSTRACT

Traffic signal control (TSC) is still one of the most significant and challenging research problems in the transportation field. Reinforcement learning (RL) has achieved great success in TSC but suffers from critically high learning costs in practical applications due to the excessive trial-and-error learning process. Offline RL is a promising method to reduce learning costs whereas the data distribution shift issue is still up in the air. To this end, in this paper, we formulate TSC as a sequence modeling problem with a sequence of Markov decision process described by states, actions, and rewards from the traffic environment. A novel framework, namely TransformerLight, is introduced, which does not aim to fit into value functions by averaging all possible returns, but produces the best possible actions using a gated Transformer. Additionally, the learning process of TransformerLight is much more stable by replacing the residual connections with gated transformer blocks due to a dynamic system perspective. Through numerical experiments on offline datasets, we demonstrate that the TransformerLight model: (1) can build a high-performance adaptive TSC model without dynamic programming; (2) achieves a new state-of-the-art compared to most published offline RL methods so far; and (3) shows a more stable learning process than offline RL and recent Transformer-based methods. The relevant dataset and code are available at Github¹.

*Contributed equally to this research

†Linyuan Lü is the corresponding author

¹<https://github.com/Smart-Trafficlab/TransformerLight>



This work is licensed under a Creative Commons Attribution International 4.0 License.

KDD '23, August 6–10, 2023, Long Beach, CA, USA

© 2023 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0103-0/23/08.

<https://doi.org/10.1145/3580305.3599530>

CCS CONCEPTS

• **Computing methodologies** → **Computational control theory**; • **Applied computing** → **Transportation**.

KEYWORDS

traffic signal control, reinforcement learning, sequence modeling, dynamic programming, Transformer

ACM Reference Format:

Qiang Wu, Mingyuan Li, Jun Shen, Linyuan Lü, Bo Du, and Ke Zhang. 2023. TransformerLight: A Novel Sequence Modeling Based Traffic Signaling Mechanism via Gated Transformer. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '23)*, August 6–10, 2023, Long Beach, CA, USA. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3580305.3599530>

1 INTRODUCTION

Traffic signal control (TSC) is still one of the primary approaches to enhance the efficiency of urban traffic flow [29]. However, the optimization of TSC is challenging due to the stochasticity and non-linearity of the complex traffic system. Now, most common TSC systems rely heavily on predefined operations on traffic signal settings by human experts [13, 20, 27], which are not adapted for dynamic traffic flows in real time. Hence, it is crucial to develop novel mechanisms to ease traffic congestion and enhance transportation efficiency beyond these traditional TSC methods.

Machine learning (ML) frameworks, especially the Deep learning (DL) approaches, learn automatically from the observed data, achieving tremendous success in a wide range of domains. But the TSC problem is not a typical ML problem conducive to the training and testing process on specified data since the traffic system is continually changing in the real world. Reinforcement learning (RL) can learn and adjust policies from the feedback of the traffic environment, hence, when empowered by DL, RL-related methods have also gained some good results for TSC [28, 34, 36], though

often more theoretically or tested in lab only simulations. Unfortunately, these RL-related approaches, in contrast, require an unstable and slow training process before they can be deployed at unobserved intersections. Moreover, most training processes are conducted in a simulation environment considering the safety of real traffic. To our best knowledge, few traffic systems have adopted RL-related methods for TSC in the real world.

A critical constituent of a conventional RL model's training process is the trial-and-error, but offline RL shifts the emphasis to previous interaction data and learns policies without using the trial-and-error process [18]. Offline RL solutions have been found to be beneficial in overcoming the limitations of traditional RL methods. However, offline RL faces the problem of data distributional shifts that might cause overestimation of the policy values, while online RL approaches can address these issues with appropriate exploration strategies. Despite the fact that recent offline RL models [18] have gained promising performance, the out-of-distribution (OOD) can be partially mitigated by constraining the decision policies [9] or underestimating policy values [16].

Now, Transformers [26] have been successfully used for a variety of natural language processing (NLP) applications, such as Bert [6] and GPT [2]. Many researches have also discovered that transformers have significantly advanced the computer vision (CV) fields [7]. Although most NLP and CV tasks are supervised or semi-supervised by data labels, the conventional RL challenge for decision-making by reward mechanism, making applications of transformers in RL comparatively under-investigated. Recently, Decision Transformer [4] and Trajectory Transformer [14] are trained on offline datasets (collected from experiences), utilizing a sequence modeling rather than a policy as used in traditional RL techniques such as temporal difference (TD) learning. These Transformer models have demonstrated superior performance compared to state-of-the-art (SOTA) offline RL techniques in various decision-making systems but have not yet been applied in TSC tasks.

The limitations of the current TSC methods are summarized as follows: (1) Traditional RL methods have a critically high learning cost in practical applications due to the trial-and-error learning processes. (2) Offline RL approaches still suffer from data distribution shift issues with dynamic programming. (3) Transformer models have not yet been applied in TSC tasks for proper traffic signal decision-making.

To tackle the above challenges, we summarize our contributions as follows:

- We innovatively formulate TSC as a sequence modeling problem with a sequence of Markov decision process (MDP) described by states, actions, and rewards, based on transformer architecture.
- We build a high-performance offline TSC model without involving dynamic programming.
- We design TransformerLight that achieves a new SOTA compared to most offline RL methods and shows a more stable learning process.

Our work is an essential step towards reducing the obstacle of complexity to deploying artificial intelligence (AI) based TSC methods in the real world.

2 RELATED WORKS

In this section, we briefly introduce conventional and RL-based TSC methods and recent models of sequence modeling for decision-making systems.

2.1 Conventional approaches

Numerous TSC methods have been proposed for efficient traffic volume. Still, one of the widely used approaches is to set fixed-time plans for the cycle sequence of the traffic signal phases, which was proposed back in 1958 [27]. Since fixed-time plans cannot adapt to traffic flow dynamics, smarter methods in transportation engineering have been proposed to design signal regulations under traffic dynamics. Applying thresholds to determine whether the signal should change is one of the most common approaches [5]. The signal will change if the queue length of the vehicles in one traffic signal is larger than a threshold. SCOOT [13] and SCATS [20] were developed to select from manually designed signaling plans (phase offset and signal cycle duration) under the assumption of certain traffic conditions. As a result, these methods rely heavily on the knowledge of experts regarding the predetermined thresholds or traffic conditions.

2.2 RL-related methods

2.2.1 Online RL. Lately, RL methods have vastly outperformed traditional methods by using trial-and-error procedures in TSC. These approaches often input the traffic state in real-time and learn a policy to manage the signals. These studies vary in two aspects: models and conditions (state, reward) for RL.

Models for RL: Current RL solutions used value-based (Q-learning) [30], policy-based (policy gradient) [22], and policy & value-based (actor-critic) [31] methods for the optimization of TSC.

Conditions for RL: Various traffic state features have been applied for RL state or reward, such as queue length [25, 32], number of vehicles [28], and average travel time [34, 36]. Moreover, some recent works just focused on traffic state to enhance the results of the RL-based methods. Efficient pressure (EP) [32] used queue length to capture the relationship between upstream and downstream lane capacity, achieving new SOTA without changing the structures of the RL models. Then, considering the requirements for running vehicles, Advanced-MP [34] has further gained outstanding results after EP. Meanwhile, different types of rewards were adapted by RL-related methods, including average travel time [28, 34, 36], queue length [30], etc.

In general, online RL approaches now require a carefully designed RL structure, many training samples, and a lengthy training process, which may be relatively challenging in the actual deployment. Moreover, designing complex state or reward heavily relies on empirical knowledge about traffic.

2.2.2 Offline RL. In contrast to conventional (online) RL algorithms, offline RL shifts the trial-and-error training mode and directly learns an appropriate policy from offline data without environmental interactions [18]. New advancements have recently illustrated the benefits of data-driven policy learning in offline RL techniques [9, 10, 15, 16]. Offline RL is still a challenging research topic because the learning algorithm must rely on the static

dataset only, and there is no proper way to enhance exploration. This OOD issue can be mitigated by constraining the decision policies (policy-based) [9] or underestimating policy values (value-based) [16]. Policy-based methods for offline RL in the class of regularized policies rely on behavior cloning for policy regularization [9, 15]. Value-based approaches for offline RL are achieved by adding a regularization term to the original Bellman error objective for the lower bound of the real Q function [16]. In addition to OOD issues, offline RL training has a larger variation than transformers model training. Because it is essential to road safety, TSC requires offline methods to lower expensive trial-and-error training in the real world.

2.3 Sequence modeling for decision-making

With the development of sequence modeling technologies, Long Short-Term Memory (LSTM) [12], sequence-to-sequence (Seq2Seq) [11], and Transformer [26] methods have succeeded widely. Transformers based on self-attention mechanisms have lately achieved ground-breaking results in NLP [2] and CV [7] tasks, because of their capacity to scale to vast volumes of data and simultaneously integrate information successfully across lengthy time horizons.

Since the RL problem is also concerned with sequences of MDP, it is tempting to wonder if sequence modeling methods would boost the performance of RL. Recent Decision Transformer (DT) [4] recasts the offline RL problem as one of conditional sequence modeling, which is in contrast to traditional RL methods relying on proper value functions or computing policy gradients. Meanwhile, Trajectories Transformer (TT) [14] explores how offline RL can be tackled with the tools of sequence modeling, using a Transformer architecture to model distributions over trajectories and repurposing beam search as a planning algorithm. In light of these works, Transformer architecture has broad prospects in decision-making tasks. To the best of our knowledge, few Transformer architectures have been directly used to make decisions for TSC.

TSC studies based on AI technologies [23, 28, 34] are constantly presented at prestigious database or ML conferences in order to co-develop efficient TSC techniques for improving transportation. However, the limitations of these AI technologies are costly and time-consuming. Offline RL is a promising approach, but it needs to address OOD issues via dynamic programming.

In this paper, we aim to build the first Transformer-based model to lower the expensive trial-and-error training process and enhance the performance and stability of the offline model for TSC as a novel contribution to the relevant research and industry community.

3 PRELIMINARY

In this section, we first briefly introduce relevant definitions of TSC and the Transformer model. Then, present the problem description of this paper.

3.1 Definitions of TSC

To better demonstrate the pertinent concepts, we use the typical example of a four-way intersection, as shown in Figure 1. We summarize the definitions of TSC as follows:

- **Traffic network:** the intersections are denoted by $(I_1, ..., I_N)$, connected by roads as a network, and each road consists of lanes.
- **Traffic movement:** a vehicle travels from the intersection's entrance lane $lane_{in}$ to the departure lane $lane_{out}$, indicated by the notation $TM = (lane_{in}, lane_{out})$.
- **Traffic signal phase:** traffic movement pairs without conflicts are grouped into a traffic signal phase, denoted as: $s = \{(TM_1), (TM_2)\}$, where TM_1 and TM_2 are two traffic movements.

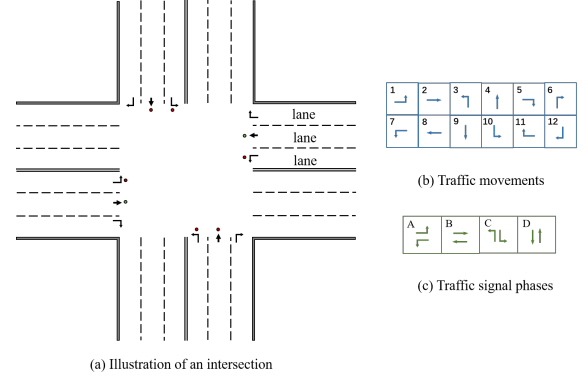


Figure 1: Illustration of a 4-approach intersection with four traffic signal phases. There are twelve traffic movements (1 to 12) and four traffic signal phases (A to D).

3.2 Transformer

The transformer framework [26] is essentially an Encoder-Decoder structure consisting of stacked self-attention layers with residual connections. In Transformer's encoder, the input data will first pass through a module called 'self-attention' to obtain a feature vector Z with additional weights. The self-attention layer maps the input token to vectors, including key K , query Q , and value V , using linear transformations.

$$Z_{att} = Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V \quad (1)$$

where Z_{att} is the attention embedding, $score = QK^T$ and $\sqrt{d_k}$ is the score normalization for stable gradient. Then, Z_{att} is sent to the next module of the encoder, feed-forward neural network (FFN), with two layers of fully-connect neural networks activated by the Relu function.

$$Z = FFN(Z_{att}) \quad (2)$$

In addition, the Transformer introduces the feature of position encoding (Position Embedding) when encoding input vectors. Thus, the Transformer can distinguish input tokens in different positions.

In this work, we modify the transformer architecture with a gated self-attention layer for regressive generation by predicting tokens only using previous tokens in the sequence, similar as the GPT architecture [2].

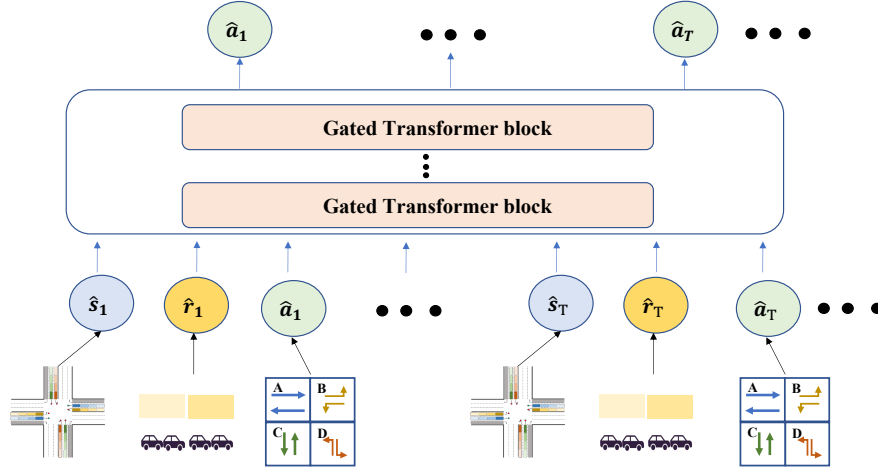


Figure 2: The architecture of TransfomerLight consists of stacked gated transformer blocks based on a GPT model.

3.3 Problem description

The problem is multi-intersection TSC to optimize traffic flow by adjusting the phase of traffic signals. We consider a decentralized model that one controller optimizes its intersection's traffic signal. The controller first models trajectory (τ) TSC data as an unstructured sequence consisting of states, actions, and rewards.

$$\tau = (s_1, a_1, r_1, \dots, s_T, a_T, r_T) \quad (3)$$

where T is time-step, (s_t, a_t, r_t) is the traffic state, signal action, and reward from the experience of RL agents with interaction with traffic environments. Generally, the traffic states s_t include the number of vehicles (NV), the queue length (QL), the efficient traffic movement pressure (EP), effective running vehicles (ERV), the traffic movement pressure (TMP), etc. Meanwhile, the traffic reward r_t consists of QL, TMP, average travel time (ATT), etc.

Herein, the goal is to build a sequel model $\mathcal{M}$ for each intersection in order to learn meaningful patterns and conditionally generate actions (optimal policy) after training in offline datasets. Thus, it is possible to minimize the amount of ATT that vehicles spend in the traffic network out of the offline datasets, denoted as:

$$\min ATT_i^{Inter_i(M_i)} \quad (4)$$

where $ATT_i^{Inter_i(M_i)}$ is the average time vehicles spending at intersection $Inter_i$ based on a sequel model M_i .

4 METHODS

In this section, we introduce TransfomerLight, a carefully designed transformer architecture for TSC based on a gated transformer architecture by auto-regressively modeling interactions of the TSC trajectories.

4.1 Model TSC Trajectories

We first model TSC trajectory τ as an unstructured sequence as shown in Equation 3, consisting of states, actions, and rewards, as problem description defined in section 3. Then, we normalized for different traffic states (NV, QL, EP, EVP, TMP and etc.) and rewards

(QL, TMP, and etc.), separately. Thus, the normalized $\hat{\tau}$ is:

$$\hat{\tau} = (\hat{s}_1, \hat{a}_1, \hat{r}_1, \dots, \hat{s}_T, \hat{a}_T, \hat{r}_T) \quad (5)$$

4.2 Architecture

We design the TransfomerLight architecture with gated mechanisms and self-attention blocks based on the GPT [2] framework (as shown in Figure 2).

4.2.1 Gated transformer block. Our transformer architecture with gated transformer block can further stabilize the learning process by leveraging various architectures, which is motivated by LSTM [12] and GTrXL [24].

The vanilla transformer block consists of two modules: multi-head self-attention (MHSA) and feed-forward neural network (FFN). We add gated layer g to MHSA and FFN modules respectively (as shown in Figure 3).

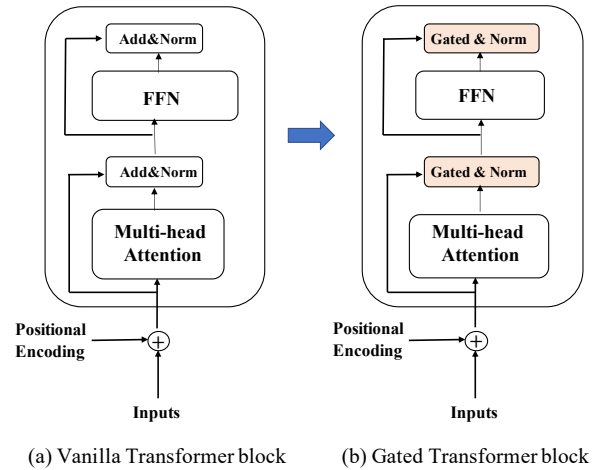


Figure 3: Structures of transformer block: (a) vanilla transformer block, (b) gated transformer block.

Here, we formulate the gated transformer block in two steps:

(1) Gated MHSA:

$$Z_{att}^g = g_{MHSA}(softmax(\frac{QK^T}{\sqrt{d_k}})V, X) \quad (6)$$

where g_{MHSA} is a gating layer function for MHSA, X is the input embedding generated from normalized trajectory $\hat{\tau}$ and the position embedding pos . And key K , query Q , and value V are generated from X by different weights. pos is the position embedding. After we get the embedding of MHSA Z_{att}^g , we put it forward to gated FFN.

(2) Gated FFN:

$$Z^g = g_{FFN}(FFN(Z_{att}^g), Z_{att}^g) \quad (7)$$

where g_{FFN} is a gating layer function for FFN, Z^g is the output embedding of a gated transformer block.

Specifically, we design a gated function, connecting both input streams: output stream o from module and residual stream r , with a Sigmoid function σ :

$$g(o, r) = \sigma(W_g o + b_g) \odot o + (1 - \sigma(W_g o + b_g)) \odot r \quad (8)$$

where (o, r) is the input of the gated function. W_g and b_g are the parameters for the gated function.

4.2.2 TransformerLight. We pass TransformerLight a normalized trajectory $\hat{\tau}$, containing tokens of the last T timesteps for the tuples of state, reward, and action. The TransformerLight consists of three modules: Input, Gated transformer blocks, and Output (As shown in Table 1).

Table 1: Network structure design of TransformerLight

Module	Layer (input size, output size)
Input Module	MLP (location dim, hidden size)
	MLP (state dim, hidden size)
	MLP (reward dim, hidden size)
	MLP (action dim, hidden size)
	Stack (hidden size, 3 x hidden size)
	LN(3 x hidden size, 3 x hidden size)
Gated Transformer Module (n blocks)	LN(3 x hidden size, 3 x hidden size)
	Gated MHA (3 x hidden size, 3 x hidden size)
	Sigmoid (3 x hidden size, 3 x hidden size)
	LN(3 x hidden size, 3 x hidden size)
	MLP (3 x hidden size, 3 x hidden size)
Output Module	Sigmoid (3 x hidden size, 3 x hidden size)
	MLP (3 x hidden size, action dim)
	Tanh (action dim, action dim)

Input module: Four different multilayer perceptron (MLP) layers from raw inputs with original dimensions (*state dim*, *reward dim*, *action dim*, and *location dim*), obtaining token embeddings for the state, action, reward, and location separately. Moreover, standard positional embedding is applied to a tuple of a state, reward, and action in one timestep. Then, we concat the embeddings of state, reward, and action by adding location embedding together to generate a dimension of 3 x hidden size matrix.

After that, layer normalization (LN) [1] normalizes these tuples of token embeddings for input embedding X .

Gated transformer blocks module: A GPT model [2] with our gated transformer blocks will process these embeddings to make autoregressive predictions for future action. We set an LN layer to process the input embedding X before putting it into Gated MHSA. After the Sigmoid function and another LN layer, the Gated FFN is implemented by an MLP.

Output module: An MLP layer with Tanh activation for the action prediction.

4.3 Algorithm

As previously stated, normalized $(\hat{s}_t, \hat{a}_t, \hat{r}_t)$ from $\hat{\tau}$ represents the traffic state, signal action, and reward from RL agents' historical interactions with traffic environments. We use various states in TSC trajectory data, apply four traffic signal phases $\{A, B, C, D\}$ (as shown in Figure 1 (c)) for action space to choose, and choose different rewards in TSC trajectory data.

We summarize the TransformerLight algorithm as Algorithm 1.

Algorithm 1 TransformerLight

Input: Normalized trajectories set $[\hat{\tau}]$ with tuples of $(\hat{s}, \hat{a}, \hat{r})$
Initialize: Parameters: W_K, W_Q, W_V, W_g and $b_g > 0$

```

1: for  $\hat{\tau}$  in set  $[\hat{\tau}]$ : do
2:   for  $(\hat{s}, \hat{a}, \hat{r})$  in  $\hat{\tau}$ : do
3:     {# Generate position embedding at time step  $t$ }
4:      $pos_{emb} = embed_{pos}(t)$ 
5:     {# Generate embeddings for  $(\hat{s}, \hat{a}, \hat{r})$ }
6:      $\hat{s}_{emb} = MLP_s(\hat{s}) + pos_{emb}$ 
7:      $\hat{r}_{emb} = MLP_r(\hat{r}) + pos_{emb}$ 
8:      $\hat{a}_{emb} = MLP_a(\hat{a}) + pos_{emb}$ 
9:     {# Generate input embedding  $X$ }
10:     $X = concat(\hat{s}_{emb}, \hat{r}_{emb}, \hat{a}_{emb})$ 
11:    {# In our gated transformer:}
12:    for block in all gated transformer blocks: do
13:       $K = W_K X; Q = W_Q X; V = W_V X;$ 
14:       $Z_{att} = softmax(\frac{QK^T}{\sqrt{d_k}})V$ 
15:       $Z_{att}^g = \sigma(W_g Z_{att} + b_g) \odot Z_{att} + (1 - \sigma(W_g Z_{att} + b_g)) \odot X$ 
16:       $Z^g = g_{FFN}(FFN(Z_{att}^g), Z_{att}^g)$ 
17:    end for
18:    {# Select output embedding for action prediction}
19:     $a^{pred} = pred_{action}(Z^g)$ 
20:    {# Compute the Cross-Entropy: }
21:     $Loss_{CE} = - \sum_k a^{predict} \log(a^{true})$ 
22:    Updated parameters  $W_K, W_Q, W_V, W_g$  and  $b_g$ 
23:  end for

```

4.4 Discussion

What are the benefits of using a sequence modeling for decision-making for TSC?

We summarize three main benefits: (1) For better decision-making

results: Sequence modeling models have demonstrated their ability to make decisions that could match or outperform the current offline RL methods. Our TransformerLight also achieves the SOTA results in TSC. (2) For a stable training process: Research has shown more stable training of transformers in the high-variance RL setting. Moreover, a large body of well-established research studies stable training technologies of transformer models. (3) To avoid OOD issues: The OOD issues are still up in the air for offline RL, while TransformerLight avoids the need for regularization or conservatism for objectives because it is not necessary to optimize with complex objective functions.

What are the main differences of TransformerLight compared to recent Transformer-based methods [4, 14] in decision-making system?

(1) Decision Transformer [4] feeds the model with a "returns-to-go" reward in sequence. In contrast, TransformerLight directly applies a reward signal from the trajectory. Moreover, TransformerLight designs a gated transformer block for a more stable and efficient training process. (2) Trajectory Transformer [14] uses Beam Search for planning, which can be repurposed for control, focusing on imitation learning, goal-conditioned reinforcement learning, and offline reinforcement learning. Unlike the Trajectory Transformer, TransformerLight mainly applies a Transformer-based architecture with minimal changes. From the experimental results (detailed in Section 5), Trajectory Transformer is not suitable for the TSC task.

Additionally, our TransformerLight is not limited to TSC tasks but is also available for various offline-RL tasks. TransformerLight can be used to optimize the decision-making process of a robot by considering a sequence of actions and their potential outcomes, similarly, in the context of autonomous driving, TransformerLight can enable vehicles to make intelligent decisions based on a sequence of observations and actions.

Why gated Transformer performs better than the vanilla version?

We design a Gated Transformer (GT) block that incorporates gating mechanisms, considering three major advantages of the GT over the vanilla version. (1) Improved gradient flow: The gating mechanisms in our GT allow for better gradient flow during back propagation. (2) Enhanced expressivity: The gating mechanisms provide the GT with additional expressivity, allowing it to learn richer and more complex representations. (3) Better handling of noise: Our GT can better handle noisy input data by selectively suppressing irrelevant information.

Why do offline-RL methods outperform some online-RL methods such as FRAP or MPLight?

Offline RL methods can outperform online RL methods in certain scenarios due to several reasons. (1) They can learn from expert demonstrations, quickly acquiring useful knowledge and improving their policy. (2) Offline RL methods can leverage a large dataset collected in advance, reducing sample complexity and leading to faster convergence. (3) Offline methods can achieve a better exploration-exploitation trade-off by using a diverse dataset. However, the performance of offline RL methods compared to online RL methods depends on the specific problem, dataset quality and size, and the algorithms employed.

5 EXPERIMENTS

In this section, comprehensive experiments are conducted based on multiple real-world datasets to evaluate our TransformerLight.

5.1 Datasets

The datasets are divided into two categories: real-world traffic flow with road topologies and historical trajectories with traffic environment interactions. We apply seven real-world traffic datasets in three cities of different sizes in two countries (namely, Jinan, Hangzhou, New York) [36]. They are widely used as benchmarking datasets for TSC problem.

Jinan datasets: The road network contains 12 (3×4) intersections with three traffic flow datasets ($Jinan_1, Jinan_2, Jinan_3$).

Hangzhou datasets: The road network contains 16 (4×4) intersections with two traffic flow datasets ($Hangzhou_1, Hangzhou_2$).

New York datasets: The road network contains 192 (28×7) intersections on the road networks with two traffic flow datasets ($NewYork_1, NewYork_2$).

Moreover, the historical trajectory data is recorded from various online RL methods [3, 28, 36] to interact with the traffic simulator CityFlow [33]. These methods correspond to different frameworks, traffic states, and rewards.

The offline dataset is generated by different policies, similar to how the data buffer of off-policy RL (DDPG [19]) is generated. Thus, the trajectory dataset, noted as $[\tau]$, covers different traffic states (NV, QL, TMP). Also, $[\tau]$ has different rewards (QL, TMP). Finally, we record two groups (five offline trajectory datasets: $Jinan^1, Jinan^2, Jinan^3, Hangzhou^1$, and $Hangzhou^2$) for cities of Jinan and Hangzhou. All offline RL-based and transformer-based methods tested in this paper, apply this trajectories' data to train their models.

5.2 Evaluation Metrics

We use the average travel time (ATT) as the main metric for evaluation in this experiment. It calculates all the vehicles that spent the time between entering roads and leaving the roads in 3600 seconds, which is the most frequently used measure of performance to control traffic signals.

5.3 Compared Methods

We compare our method with the following benchmark methods, including traditional online RL and recent offline approaches. All the RL methods are trained with a 60-minute simulation episode. The final result is the average of three independent results from the test.

Traditional RL Methods: The traditional RL Methods consist of FRAP [36], AttendLight [23], CoLight [28], Efficient-MPLight [32], Advanced-CoLight [34], PRGLight [35] and AttendLight [23]. Notably, PRGLight [35] and AttendLight [23] are not employed to generate the historical trajectory data, which makes the comparison fair.

Behavior Cloning (BC): We employ Behavior Cloning (BC) to provide the performance of a pure imitative method.

Offline RL Methods: We apply the latest offline RL method CQL [16], TD3+BC [8], and BEAR [15] for comparison.

Transformer-based Methods: We use Decision Transformer [4], Trajectory Transformer [14], and Algorithm

Table 2: Performance comparison of different methods based on real-world datasets (ATT in seconds, the smaller the better).

Type	Method	Jinan			Hangzhou		New York	
		1	2	3	1	2	1	2
Online-RL	FRAP	299.56	268.57	269.20	308.73	355.80	1192.23	1470.51
	MPLight	297.68	274.32	268.00	313.16	355.35	1321.40	1642.05
	CoLight	271.17	251.22	248.87	300.07	339.76	1065.64	1367.54
	Efficient-MPLight	261.81	241.35	238.80	284.49	321.08	1301.83	1485.52
	Advanced-MPLight	258.29	234.78	231.76	273.26	312.68	1198.64	1439.26
	Advanced-CoLight	245.73	232.63	229.01	270.45	310.74	970.05	1300.62
	AttendLight	277.53	250.29	248.82	293.89	345.72	1586.09	1683.57
	PRGLight	291.27	257.52	261.74	301.06	369.98	1283.37	1472.73
Offline-RL	BC	282.31	243.52	241.85	364.01	341.82	1132.30	1460.94
	TD3-BC	284.70	242.30	258.56	284.77	344.88	1186.08	1536.76
	BEAR	271.29	242.43	254.77	285.57	359.62	1033.63	1425.37
	CQL	265.68	260.57	234.17	287.31	338.52	1099.83	1405.77
Transformer-based	Decision Transformer	274.79	244.19	254.45	284.02	372.01	1091.11	1463.83
	Trajectory Transformer	722.00	807.90	673.57	680.19	527.44	1854.39	1987.43
	Algorithm Distillation	268.21	242.81	251.10	281.78	379.08	1129.98	1453.77
	TransformerLight	256.16	235.78	232.39	273.18	313.91	1097.33	1413.25

Distillation [17] to train on our historical trajectory dataset for final results of ATT.

5.4 Settings

All benchmarks of TSC methods are trained using various hyperparameters such as learning rate, sample size, which reproduce the results of ATT in their original works. Additionally, because benchmarks of offline methods have yet been applied to the TSC problem, we use their released code on our historical trajectory dataset.

We first compute the average of the most recent ten episodes with a 60-minute simulation. Then, the final ATT result is the average of three independent results of ten episodes. We pragmatically set phase duration as 15s and signal phase as 4-phase for all methods. A three-second yellow signal follows each green signal. Also, we set hyperparameters of TransformerLight in our released code.

5.5 Overall Results

We divide all methods into three categories: online-RL, offline-RL, and Transformer-based. To obtain TSC results, we conduct experiments on real-world datasets (Jinan, Hangzhou, New York) of traffic topology and flow for online-RL methods. Then, offline-RL and Transformer-based methods are first trained on the historical trajectory data and directly deployed on the real-world datasets. Table 2 reports our experimental results for Jinan, Hangzhou, and New York real-world datasets with respect to the evaluation metrics (ATT). The bold black numbers are the best performance in Online-RL, Offline-RL, and Transformer-based, respectively. From the overall results (as shown in Table 2), we have the following findings: (1) Advanced-CoLight, CQL, and TransformerLight perform the best in online-RL, offline-RL, and Transformer-based categories, respectively. (2) TransformerLight consistently performs better on five in seven datasets than recent offline approaches

(offline-RL and Transformer-based categories). However, the remaining two datasets are also close to the best results in offline approaches. However, Trajectory Transformer has shown a performance worse than expected. It is demonstrated that not all recent Transformer-based models are adapted to the TSC tasks. (3) The performance of TransformerLight overpass most of the TSC methods in most TSC methods in all categories and matches the SOTA online-RL: Advanced-CoLight. (4) We reproduced the code according to the paper for the Trajectory Transformer method. The results were not as satisfactory as expected, probably because the beam search for the planning algorithm is unsuitable for TSC tasks.

Hence, our TransformerLight with gated transformer blocks has shown mense potential for reducing the obstacle of complexity to deploying AI-based TSC methods in the real world.

5.6 Model Analysis

5.6.1 Model Generalization . The Advanced-MPLight has shown the best transferability among online-RL methods [34]. Meanwhile, CQL outperforms other offline-RL approaches [16]. Thus, we choose Advanced-MPLight and CQL as benchmark methods for model generalization experiments to compare with our TransformerLight. Advanced-MPLight is an online-RL method trained on a dataset of Hangzhou-1 with environment interactions. Meanwhile, CQL and TransformerLight are offline methods only trained on 20% historical trajectory data, which are generated in *Hangzhou₁* real-world dataset. Then, these methods are directly transferred to other real-world datasets. At last, we compare the performance of these transferred methods. As shown in Figure 4, we can see that TransformerLight has an advantage in terms of transferability. Especially in large-scale data (*NewYork₁* and *NewYork₂*), TransformerLight shows a significant improvement in model generalization.

In addition, we apply the Advanced-MPLight, which is directly trained on the historical trajectory data $Jinan^1(JN^1)$ without environment interactions, and then transferred to other cities. The generalization performance of Advanced-MPLight drops sharply compared to TransformerLight.

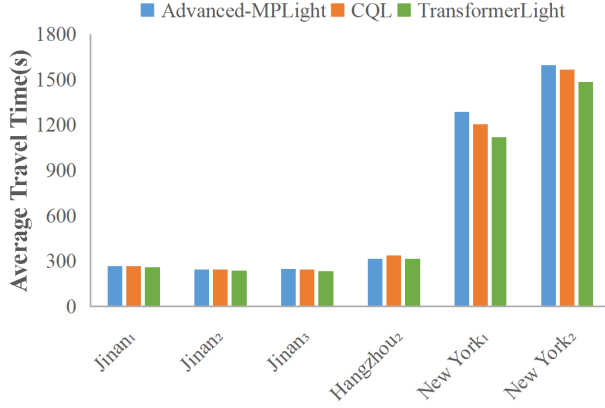


Figure 4: Performance comparison of different methods trained on $Hangzhou_1$, then directly transfer to other datasets (ATT in seconds).

5.6.2 Model Stability. We choose Advanced-CoLight (AC) [34], DecisionTransformer (DT) [4], TrajectoryTransformer (TT) [14], Algorithm Distillation (AD) [17], and our TransformerLight (TL) to compare the stability of the learning process. Because these methods have shown the excellent performance in online-RL and Transformer-based methods respectively. Each method runs for 30 episodes with evaluations on every three episodes, where each evaluation reports the average travel time over one episode.

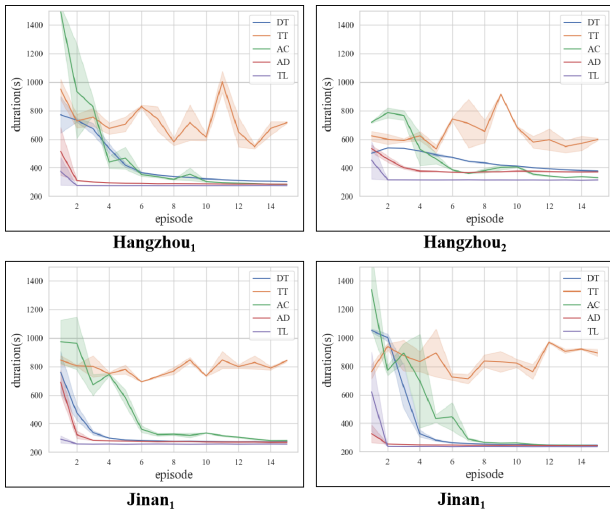


Figure 5: Comparison of learning curves in four datasets for typical TSC methods.

As shown in Figure 5, we can see that TransformerLight quickly converges in the first three training episodes and shows exceptionally stable and uniformly smooth learning curves in most of the real-world datasets. On the other hand, compared methods need a relatively long and unstable learning process.

5.6.3 Model Speed. We compared the running speed among typical TSC models and offline methods by computing the average interval between the input traffic state and output control action in an episode under the same computing resources. GPU devices can accelerate the transformer-based methods, while online and offline RL models need CPU devices to compute Bellman backups. As shown in Table 3, TransformerLight is 5.9 times faster than Advanced-CoLight in $Jinan_1$ dataset, although their performances in experiments are close. Meanwhile, TransformerLight is 4.6 times faster than CQL for model speed and outperforms CQL by 28.8% for model performance.

Table 3: Comparison on running speed (millisecond).

Method	Running Time
MPLight	135.55
CoLight	155.22
Advanced-CoLight	99.21
CQL	77.31
TD3-BC	204.05
BEAR	211.31
TL-without-gatedMHSA	23.95
TL-without-gatedFFN	24.34
Decision Transformer	35.62
TransformerLight	16.56

5.6.4 Ablation experiment. We conducted ablation experiments for providing a comparison between the designed gated Transformer block against the vanilla Transformer block. The vanilla transformer block consists of two modules: multi-head self-attention (MHSA) and feed-forward neural network (FFN). While the TransformerLight adds gated layer to MHSA and FFN modules, respectively. We removed the gated layer from MHSA and FFN modules, respectively, which produce the TL-without-gatedMHSA and TL-without-gatedFFN models. Meanwhile, we removed all the gated layers, which exactly form the Decision Transformer model. Accordingly, we compared the performance of these methods ATT score from ablation experiments.

As shown in Table 4, we find that our TransformerLight outperformed all models with various vanilla Transformer blocks in three different cities.

Table 4: Performance comparison (ATT in seconds).

Method	JN_1	HZ_1	NY_1
TransformerLight	256.16	273.18	1097.33
TL-without-gatedMHSA	258.78	273.58	1109.83
TL-without-gatedFFN	259.17	274.76	1098.62
Decision Transformer	274.79	284.02	1091.11

6 CONCLUSION

In this paper, we formulate TSC as a sequence modeling problem and introduce a framework, namely TransformerLight, generating the best possible actions by using a gated Transformer. Our method needs not fit value functions or compute policy gradients, which is more stable and suitable to be applied in real practice without dynamic programming. The experimental results demonstrate that our method achieves a new SOTA compared to most offline RL methods.

In the future, we will continue to optimize our TransformerLight model by decreasing the model scale for easier deployment. TSC task can be regarded as an offline multi-agent cooperation problem [21] which leads to a new research direction.

ACKNOWLEDGEMENTS

This work is supported by the National Natural Science Foundation of China (Grant Nos. 61673150, 11622538, T2293771), and the New Cornerstone Science Foundation through the XPLOER PRIZE.

REFERENCES

- [1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. 2016. Layer Normalization. *arXiv:1607.06450* [stat.ML]
- [2] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- [3] Chacha Chen, Hua Wei, Nan Xu, Guanjie Zheng, Ming Yang, Yuanhao Xiong, Kai Xu, and Zhenhui Li. 2020. Toward a thousand lights: Decentralized deep reinforcement learning for large-scale traffic signal control. In *Proceedings of the AAAI Conference on Artificial Intelligence*. 3414–3421.
- [4] Lili Chen, Kevin Lu, Aravind Rajeswaran, Kimin Lee, Aditya Grover, Misha Laskin, Pieter Abbeel, Aravind Srinivas, and Igor Mordatch. 2021. Decision transformer: Reinforcement learning via sequence modeling. *Advances in neural information processing systems* 34, 15084–15097.
- [5] Seung-Bae Cools, Carlos Gershenson, and Bart D'Hooghe. 2013. *Self-Organizing Traffic Lights: A Realistic Simulation*. 45–55.
- [6] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *CoRR* abs/1810.04805 (2018). <http://arxiv.org/abs/1810.04805>
- [7] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. 2020. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. *CoRR* abs/2010.11929 (2020). *arXiv:2010.11929*
- [8] Scott Fujimoto and Shixiang Shane Gu. 2021. A minimalist approach to offline reinforcement learning. In *Advances in Neural Information Processing Systems*. *Advances in neural information processing systems* 34, 20132–20145.
- [9] Scott Fujimoto, David Meger, and Doina Precup. 2018. Off-Policy Deep Reinforcement Learning without Exploration. *CoRR* abs/1812.02900 (2018).
- [10] Scott Fujimoto, Herke van Hoof, and David Meger. 2018. Addressing Function Approximation Error in Actor-Critic Methods. In *International Conference on Machine Learning (ICML)*, Vol. 80. 1587–1596.
- [11] Zengyang Gong, Bo Du, Zhidan Liu, Wei Zeng, Pascal Perez, and Kaishun Wu. 2020. SD-seq2seq: A Deep Learning Model for Bus Bunching Prediction Based on Smart Card Data. In *2020 29th International Conference on Computer Communications and Networks (ICCCN)*. 1–9.
- [12] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long short-term memory. *Neural computation* 9, 8 (1997), 1735–1780.
- [13] PB Hunt, DI Robertson, RD Bretherton, and M Cr Royle. 1982. The SCOOT online traffic signal optimisation technique. *Traffic Engineering & Control* 23, 4 (1982), 190–192.
- [14] Michael Janner, Qiyang Li, and Sergey Levine. 2021. Offline Reinforcement Learning as One Big Sequence Modeling Problem. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 34. 1273–1286.
- [15] Aviral Kumar, Justin Fu, Matthew Soh, George Tucker, and Sergey Levine. 2019. Stabilizing off-policy q-learning via bootstrapping error reduction. *Advances in Neural Information Processing Systems* 32 (2019).
- [16] Aviral Kumar, Aurick Zhou, George Tucker, and Sergey Levine. 2020. Conservative Q-Learning for Offline Reinforcement Learning. In *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (Eds.), Vol. 33. 1179–1191.
- [17] Michael Laskin, Luyu Wang, Junhyuk Oh, Emilio Parisotto, Stephen Spencer, Richie Steigerwald, DJ Strouse, Steven Hansen, Angelos Filos, Ethan Brooks, et al. 2022. In-context reinforcement learning with algorithm distillation. *arXiv preprint arXiv:2210.14215* (2022).
- [18] Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu. 2020. Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems. *CoRR* abs/2005.01643 (2020).
- [19] Timothy P. Lillicrap, Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. 2016. Continuous control with deep reinforcement learning. In *International Conference on Learning Representations, ICLR, Yoshua Bengio and Yann LeCun (Eds.)*.
- [20] P. R. Lowrie. 1990. SCATS, Sydney Co-Ordinated Adaptive Traffic System: A Traffic Responsive Method of Controlling Urban Traffic. *Roads and Traffic Authority NSW* (1990), 28.
- [21] Linghui Meng, Muning Wen, Chenyang Le, Xiyun Li, Dengpeng Xing, Weinan Zhang, Ying Wen, Haifeng Zhang, Jun Wang, Yaodong Yang, et al. 2023. Offline Pre-trained Multi-agent Decision Transformer. *Machine Intelligence Research* 20, 2 (2023), 233–248.
- [22] Seyed Sajad Mousavi, Michael Schukat, and Enda Howley. 2017. Traffic light control using deep policy-gradient and value-function-based reinforcement learning. *IET Intelligent Transport Systems* 11, 7 (2017), 417–423.
- [23] Afshin Oroojlooy, Mohammadreza Nazari, Davood Hajinezhad, and Jorge Silva. 2020. Attendlight: Universal attention-based reinforcement learning model for traffic signal control. *Advances in Neural Information Processing Systems* 33 (2020), 4079–4090.
- [24] Emilio Parisotto, H. Francis Song, Jack W. Rae, Razvan Pascanu, Caglar Gulcehre, Siddhant M. Jayakumar, Max Jaderberg, Raphaël Lopez Kaufman, Aidan Clark, Seb Noury, Matthew M. Botvinick, Nicolas Heess, and Raia Hadsell. 2020. Stabilizing Transformers for Reinforcement Learning. In *Proceedings of the International Conference on Machine Learning (ICML)*. Article 694, 12 pages.
- [25] Pravin Varaiya. 2013. *The Max-Pressure Controller for Arbitrary Networks of Signalized Intersections*. 27–66. https://doi.org/10.1007/978-1-4614-6243-9_2
- [26] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems, Long Beach, CA, USA*. 5998–6008.
- [27] F. V. Webster. 1958. *Traffic signal settings*. Technical Report 39.
- [28] Hua Wei, Nan Xu, Huichu Zhang, Guanjie Zheng, Xinshi Zang, Chacha Chen, Weinan Zhang, Yanmin Zhu, Kai Xu, and Zhenhui Li. 2019. CoLight: Learning Network-level Cooperation for Traffic Signal Control. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM '19)*. 1913–1922.
- [29] Hua Wei, Guanjie Zheng, Vikash Gayah, and Zhenhui Li. 2019. A Survey on Traffic Signal Control Methods. *arXiv preprint arXiv:1904.08117* abs/1904.08117 (2019).
- [30] Hua Wei, Guanjie Zheng, Huaxiao Yao, and Zhenhui Li. 2018. Intellilight: A reinforcement learning approach for intelligent traffic light control. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2496–2505.
- [31] Qiang Wu, Jianqing Wu, Jun Shen, Bo Du, Akbar Telikani, Mahdi Fahmideh, and Chao Liang. 2022. Distributed agent-based deep reinforcement learning for large scale traffic signal control. *Knowledge-Based Systems* 241 (2022), 108304.
- [32] Qiang Wu, Liang Zhang, Jun Shen, Linyuan Lü, Bo Du, and Jianqing Wu. 2021. Efficient pressure: Improving efficiency for signalized intersections. *arXiv preprint arXiv:2112.02336* (2021).
- [33] Huichu Zhang, Siyuan Feng, Chang Liu, Yaoyao Ding, Yichen Zhu, Zihan Zhou, Weinan Zhang, Yong Yu, Haiming Jin, and Zhenhui Li. 2019. Cityflow: A multi-agent reinforcement learning environment for large scale city traffic scenario. In *The world wide web conference*. 3620–3624.
- [34] Liang Zhang, Qiang Wu, Jun Shen, Linyuan Lü, Bo Du, and Jianqing Wu. 2022. Expression might be enough: representing pressure and demand for reinforcement learning based traffic signal control. In *Proceedings of the 39th International Conference on Machine Learning*, Vol. 162. 26645–26654.
- [35] Chenguang Zhao, Xiaorong Hu, and Gang Wang. 2021. PRGLight: A novel traffic light control framework with Pressure-based-Reinforcement Learning and Graph Neural Network. In *IJCAI 2021 Reinforcement Learning for Intelligent Transportation Systems (RL4ITS) Workshop*.
- [36] Guanjie Zheng, Yuanhao Xiong, Xinshi Zang, Jie Feng, Hua Wei, Huichu Zhang, Yong Li, Kai Xu, and Zhenhui Li. 2019. Learning Phase Competition for Traffic Signal Control. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM '19)*. 1963–1972.